IoT Challenge 1

ESP32 Motion Sensor Node

Marco Abbadessa - 10854816
Alice Berta - 10889590

Academic Year 2025–2026

Contents

1 Introduction **2**

2 Hardware Design **2**

3 Software Design **3**
3.1 Low-Level Initialization . . . 4
3.2 Sensor Acquisition . . . 4
3.3 Message Formatting . . . 4
3.4 Communication Initialization . . . 4
3.5 Deep Sleep . . . 5
3.6 Wake-Up Diagnostics . . . 5

4 Timing Measurements **5**

5 Energy Consumption Estimation **6**

6 Battery Lifetime Estimation **7**

7 Improved Version of the Code **8**
7.1 Reducing the Initial Delay after `Serial.begin()` . . . 8
7.2 Lowering the Transmission Power to 2 dBm . . . 8
7.3 Replacing `delay(100)` with a Flag-Based Wait . . . 9
7.4 Increasing the Deep Sleep Duration . . . 10
7.5 Transmitting Only on Status Change . . . 10
7.6 Hardware Limitations of the ESP32 Architecture . . . 11

8 Comparison Between Original and Improved Version **11**

9 Conclusion **13**

1 Introduction

The aim of this work is to design and emulate an IoT motion sensor node for a smart-home scenario. The node is based on an ESP32 board and is required to detect human motion through a PIR sensor, while also estimating the ambient light level through a photoresistor. The collected information is then transmitted to a sink node, which can use it to support decisions such as turning on or off the lights in the room.

According to the challenge specifications, communication must be implemented through the ESP-NOW protocol. Moreover, the sink node can be considered always reachable. To reduce energy consumption, the sensor node does not operate continuously. Instead, it follows a duty-cycled strategy: after sensing and transmitting, the ESP32 enters Deep Sleep for a fixed interval, then wakes up, and repeats the same execution flow.

The output message generated by the node must follow one of the two required formats:

- `MOTION_DETECTED-LUMINOSITY:ZZZ`
- `MOTION_NOT_DETECTED-LUMINOSITY:ZZZ`

In the implemented solution, the board periodically reads the state of a PIR sensor and the light value acquired through an LDR-based circuit, combines them into the required string, transmits the result using ESP-NOW, and finally enters Deep Sleep. This design reflects the typical behavior of battery-powered IoT sensing devices, where the active phase must be kept as short as possible and most of the lifetime is spent in a low-power state.

2 Hardware Design

The hardware architecture developed in Wokwi is intentionally simple and includes only the components required by the challenge: an ESP32 development board, a PIR motion sensor, and a photoresistor used to estimate ambient brightness.

The PIR sensor is connected to GPIO13 and is used as a digital input device. Its output directly indicates whether motion is detected in the monitored environment. Since the PIR sensor provides a binary signal, it is particularly suitable for distinguishing between the two states required by the challenge, namely motion detected and motion not detected.

The ambient light level is measured through a photoresistor connected to the analog input GPIO34. In practice, the LDR is used within a voltage-divider configuration with a $10\,\mathrm{k}\Omega$ resistor, so that the ESP32 can observe a voltage that varies with light intensity. This analog value is then converted through the board ADC and used as the luminosity field of the transmitted message.

The complete node is therefore composed of:

- one ESP32 board;
- one PIR sensor connected to GPIO13;
- one photoresistor connected to GPIO34.

This hardware configuration is sufficient to satisfy the sensing requirements of the challenge while keeping the system compact and easy to analyze.

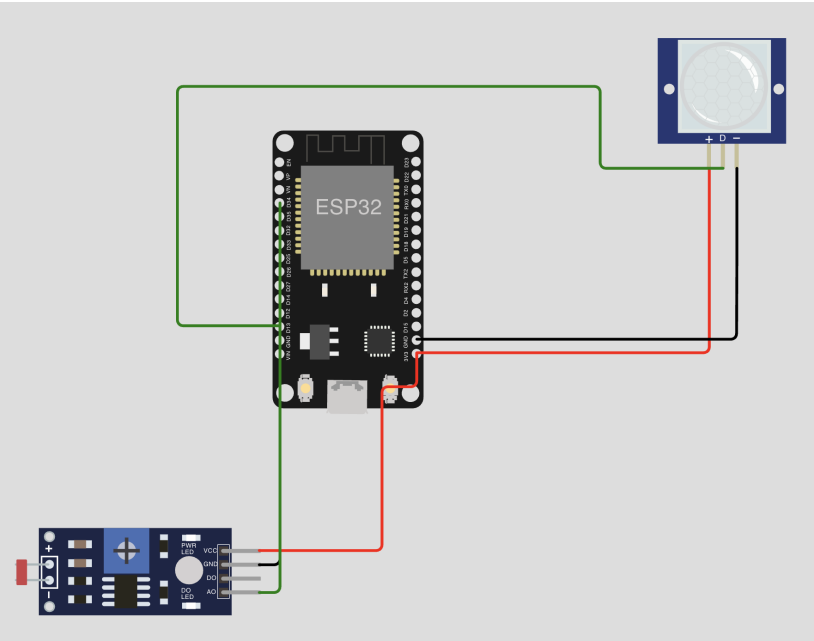


Figure 1: *Wokwi schematic of the ESP32 motion sensor node.*

3 Software Design

The software was designed according to the duty-cycled operating model required by the challenge. The core logic is not divided between `setup()` and `loop()`. Instead, the entire behavior of the node is implemented inside `setup()`, while `loop()` is intentionally left empty.

This choice is not accidental, but directly follows from the behavior of the ESP32 in Deep Sleep mode. When the board wakes up from Deep Sleep, it does not resume execution from the point where it was suspended. Instead, it performs a full reset and starts again from the beginning of `setup()`. For this reason, the `loop()` function is never actually used as the core execution point of the application.

At the beginning of each cycle, the board initializes the serial interface with `Serial.begin(115200)` and waits one second before printing diagnostic messages. This delay ensures that the serial monitor is ready before the debugging output is emitted. The code then increments and prints a boot counter and reports the wake-up reason.

The boot counter is declared as:

```
RTC_DATA_ATTR int bootCount = 0;
```

The `RTC_DATA_ATTR` attribute stores the variable in RTC memory, which is the only memory area that survives Deep Sleep on real ESP32 hardware. Therefore, on a physical board the boot counter would increase across wake-up cycles. In Wokwi, however, RTC memory is not preserved correctly across Deep Sleep cycles, so the variable may appear to reset and remain equal to 1 at each restart.

At each activation cycle, the program executes the following operations:

1. low-level initialization and diagnostic output;
2. acquisition of motion and light data from the sensors;

3. generation of the output message in the required format;
4. activation of WiFi in station mode and initialization of ESP-NOW;
5. transmission of the message to the sink node;
6. transition to Deep Sleep for the programmed time interval.

3.1 Low-Level Initialization

Some low-level ESP32-specific headers are included in the project, namely `soc/soc.h` and `soc/rtc_cntl_reg.h`. These are needed because the code directly accesses the brownout detector control register through:

```
WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);
```

The brownout detector is a hardware safety mechanism that resets the ESP32 when the supply voltage falls below a critical threshold. In this project it is disabled because enabling WiFi and ESP-NOW may produce a short current spike, especially in battery-powered operation, which can momentarily lower the voltage and trigger an undesired reset even when the system is otherwise functioning correctly. In a real commercial product this choice would require more caution, but in a simulated environment and in an experimental prototype it is a common practical workaround to avoid spurious resets during transmission.

3.2 Sensor Acquisition

The PIR sensor is read through `digitalRead()`, producing a boolean value that indicates whether motion is present. This allows the software to select the correct textual prefix for the outgoing message.

The photoresistor is read through `analogRead()`. Before the acquisition, the ADC resolution is configured to 12 bits by calling `analogReadResolution(12)`. Consequently, the measured luminosity value ranges from 0 to 4095 and can be interpreted as a numerical estimate of the ambient brightness level.

3.3 Message Formatting

Once both sensor values have been acquired, the software builds the output string according to the format required by the challenge.

```
String msg = motion
  ? "MOTION_DETECTED-LUMINOSITY:" + String(lux)
  : "MOTION_NOT_DETECTED-LUMINOSITY:" + String(lux);
```

This solution guarantees that each transmitted packet strictly follows the required syntax.

3.4 Communication Initialization

After the sensing phase, the ESP32 enables WiFi in station mode by calling `WiFi.mode(WIFI_STA)`, which is required in order to use ESP-NOW. The code then initializes the ESP-NOW subsystem, registers the transmission callback, and adds a peer associated with the broadcast address.

The broadcast MAC address is:

```
uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
```

Using `FF:FF:FF:FF:FF:FF` makes it possible to send the packet to all ESP-NOW devices within range without knowing in advance the specific MAC address of the sink node. The `peerInfo` structure is the internal ESP-NOW data structure used to register the destination peer before transmission.

An important implementation detail is that WiFi and ESP-NOW are initialized only after sensor acquisition has already been completed. This is a deliberate design choice: since radio activation is the most expensive phase in terms of energy, postponing it until it is strictly necessary helps reduce the duration of the costly active phase.

3.5 Deep Sleep

Once the packet has been sent, the node prints a final diagnostic message, configures the wake-up timer, flushes the serial buffer, and enters Deep Sleep:

```
esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);
Serial.flush(); // Flush serial buffer before sleeping
esp_deep_sleep_start(); // Enter deep sleep
```

The wake-up timer must be expressed in microseconds, hence the multiplication by the conversion factor. In the current implementation, the sleep interval is set to 4.5 seconds, coherently with the challenge formula used for the value X.

After `esp_deep_sleep_start()` is executed, the ESP32 powers down almost all of its internal components and only the RTC domain remains active. When the timer expires, the chip restarts from the beginning of `setup()`, which is why the whole execution flow is designed as a self-contained cycle.

3.6 Wake-Up Diagnostics

A utility function is used to inspect the wake-up reason through `esp_sleep_get_wakeup_cause()` and print it on the serial interface. On real hardware, after the first boot the wake-up cause should be reported as timer-based wake-up. In Wokwi, however, this information is not always reproduced faithfully, and the serial output may still show a generic message such as power-on or reset.

Nevertheless, the simulator output also reports the reset code `rst:0x8` (`TG1WDT_SYS_RESET`), which confirms that the reset is actually produced by the Deep Sleep timer.

4 Timing Measurements

In order to support the energy analysis developed later in the report, the duration of the main execution phases was measured in Wokwi using the `micros()` function, as explicitly suggested by the challenge instructions.

The measurements were repeated over multiple cycles. The first cycle is affected by cold-boot effects (a `POWERON_RESET` that requires a more complete WiFi initialization, taking approximately 216 ms instead of the nominal 196 ms). The second cycle shows an additional anomaly caused by a MAC calibration issue: the simulator does not preserve flash memory

correctly between Deep Sleep cycles, so the ESP32 finds inconsistent data and must recalibrate, taking approximately 228 ms. From the third cycle onward, the behavior becomes stable and the timing values can be considered representative of normal operation. All figures reported in this section are averaged over the stable cycles only.

The average measured durations are summarized in Table 1.

Phase	**Average duration**
Boot delay (`delay(1000)`)	1000 ms
Sensor reading	0.43 ms
WiFi + ESP-NOW initialization	196.0 ms
Transmission (incl. `delay(100)`)	99.3 ms
Deep Sleep	4500 ms
Total active time	≈1296 ms
Total cycle duration	≈5796 ms
Active fraction	≈22%

Table 1: *Average timing values measured in Wokwi (stable cycles only, cycles 1–2 excluded). The boot delay is not measured by* `micros()`.

The sensor-reading phase is highly stable, with values consistently around 429–430 μs across repeated cycles, confirming that sensing is both fast and reliable. The WiFi and ESP-NOW initialization stage is significantly longer, since enabling the radio subsystem and configuring communication introduces a substantial overhead. The transmission phase requires careful interpretation: the measured TX time does not correspond only to the actual packet transmission, because the timing window includes both the call to `esp_now_send()` and a fixed `delay(100)` introduced to ensure that the send callback completes before the node enters Deep Sleep. Therefore, the reported value of approximately 99.3 ms should be understood as a synchronization interval rather than as the pure physical transmission latency.

The boot delay of 1000 ms, executed immediately after `Serial.begin()`, is not captured by the `micros()` measurements since it precedes all timed sections. It is nonetheless real active time and is therefore included in the total figures above. Summing all active phases, the ESP32 is awake for approximately 1296 ms per cycle and spends the remaining 4500 ms in Deep Sleep, yielding an active fraction of approximately 22%.

5 Energy Consumption Estimation

The average power drawn during each operating phase was extracted from the pre-recorded power-consumption CSV files provided for the challenge. Three CSV files were analyzed, covering the sensor-reading phase (`sensor_read.csv`), the WiFi initialization and transmission phase (`sender.csv`), and the Deep Sleep phase (`deep_sleep.csv`).

To accurately isolate the consumption of distinct operational states from the raw data, a custom Python script utilizing the `pandas` library was developed. Rather than computing a simple global mean, the script applied power thresholds to filter the data traces, effectively separating overlapping states (e.g., distinguishing the high-power WiFi initialization phase from the subsequent transmission peak and baseline idle currents). The resulting mean power values associated with each phase of the duty cycle are reported in Table 2.

Phase	Avg. power	Duration	Energy
Boot delay	252.22 mW	1000 ms	0.2522 J
Sensor acquisition	238.02 mW	0.43 ms	0.0001 J
WiFi + ESP-NOW initialization	632.04 mW	196 ms	0.1238 J
Transmission (19.5 dBm)	687.93 mW	99.3 ms	0.0683 J
Deep Sleep	45.78 mW	4500 ms	0.2060 J
Total per cycle	—	5796 ms	0.6504 J

Table 2: *Energy estimation for one duty cycle. Power values are derived from the simulation CSV traces. The sensor acquisition energy is below 0.001 J and is shown as 0.000 J after rounding.*

The dominant contribution to total energy per cycle is the boot delay, which alone accounts for 0.2522 J out of 0.6504 J (nearly 39% of the total) despite being simply an idle wait. The WiFi and ESP-NOW initialization phase is the second largest consumer at 0.124 J, driven by the high power draw of the radio subsystem during startup. The Deep Sleep phase contributes 0.206 J because, although the instantaneous power is very low (46 mW), the sleep interval is long enough that its total energy is non-negligible. Sensor acquisition is almost negligible, lasting less than half a millisecond.

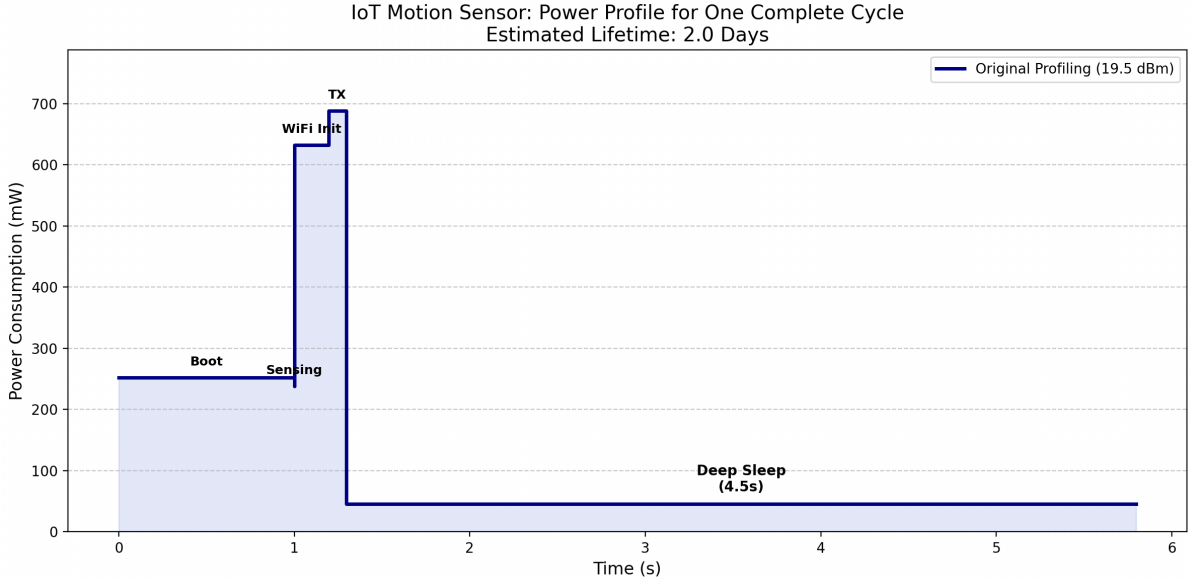


Figure 2: *Power consumption over one duty cycle*

6 Battery Lifetime Estimation

The battery energy value Y computed from the team leader's person code (10889590) is **19590** J. The total number of duty cycles supported by the battery is computed as:

$$N = \frac{Y}{E_{\text{cycle}}} = \frac{19590 \text{ J}}{0.650 \text{ J}} \approx 30115 \text{ cycles}$$

The total operating lifetime is calculated as:

$$T = N \times t_{\text{cycle}} = 30115 \times 5.796 \text{ s} \approx 174537 \text{ s}$$

Converting this value into days yields an estimated operating lifetime of:

$$T_{\text{days}} = \frac{174537 \text{ s}}{86400 \text{ s/day}} \approx 2.02 \text{ days}$$

7 Improved Version of the Code

Three concrete modifications were identified and applied to reduce energy consumption while keeping the node's functionality and communication protocol unchanged. All three are visible in the improved source code and are marked with `[IMPROVEMENT]` comments.

7.1 Reducing the Initial Delay after `Serial.begin()`

The original code waits one full second at the beginning of every cycle, immediately after initializing the serial interface:

```
Serial.begin(115200);
delay(1000); // 1 second wasted at every wake-up
```

This delay was introduced to let the serial monitor stabilize before printing diagnostic messages, and is harmless during development.

However, because the ESP32 performs a full reset at every wake-up from Deep Sleep, this line is executed at the beginning of every duty cycle, not only at the first boot. With a cycle period of approximately 5.8 seconds, this single delay accounts for more than 17% of the total cycle time and consumes 0.252 J of the 0.651 J budget per cycle, making it the single largest waste in the entire system.

In the improved version the delay is reduced from 1000 ms to 10 ms, which is sufficient to avoid serial glitches while eliminating virtually all of the wasted active time:

```
Serial.begin(115200);
// [IMPROVEMENT] Reduce initial delay from 1 s to 10 ms
delay(10);
```

Metric	Before	After
Boot energy	0.252 J	0.003 J
Energy per cycle	0.651 J	0.401 J
Estimated lifetime	2.02 days	2.72 days

Table 3: *Impact of reducing `delay(1000)` to `delay(10)`: energy per cycle drops by 0.249 J, the single largest saving among the three improvements.*

7.2 Lowering the Transmission Power to 2 dBm

The default ESP-NOW transmission power on the ESP32 is 19.5 dBm, which draws approximately 688 mW during the transmission phase.

Since the challenge specification states that the sink node is always reachable, there is no need to operate at maximum power. The transmission power is reduced to the minimum supported value of 2 dBm by adding a single API call immediately after `esp_now_init()`, made possible by including `esp_wifi.h`:

```
esp_now_init();
// [IMPROVEMENT] Reduce TX power from 19.5 dBm to 2 dBm
esp_wifi_set_max_tx_power(8); // 8 x 0.25 dBm = 2 dBm
```

At 2 dBm the transmission phase draws approximately 618 mW instead of 688 mW, reducing the instantaneous power by about 70 mW during the entire TX window. The transmission duration does not change, so the energy saving per cycle is directly proportional to this power difference.

Metric	Before	After
TX power	688 mW	617.880 mW
TX energy	0.068 J	0.061 J
Energy per cycle	0.651 J	0.644 J
Estimated lifetime	2.02 days	2.04 days

Table 4: *Impact of lowering TX power to 2 dBm: modest but entirely zero-cost saving that requires a single line of code.*

7.3 Replacing `delay(100)` with a Flag-Based Wait

After calling `esp_now_send()`, the original code waits a fixed 100 ms to ensure that the `OnDataSent` callback fires before the node enters Deep Sleep.

In practice, ESP-NOW completes the transmission in a few milliseconds, so the remainder of the delay is wasted with the WiFi radio still active. The improvement introduces a `volatile` boolean flag `messageSent`, declared at file scope, which is set to `true` by the callback as soon as the transmission is confirmed.

A polling loop then exits immediately once the flag is raised, with a safety timeout of 50 ms to prevent the node from stalling indefinitely in case of failure:

```
// [IMPROVEMENT] volatile flag instead of a fixed delay
volatile bool messageSent = false;

void OnDataSent(const wifi_tx_info_t *mac_addr,
                esp_now_send_status_t status) {
  Serial.print("[TX] Send Status: ");
  Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Ok" : "Error");
  // [IMPROVEMENT] signal that transmission has ended
  messageSent = true;
}

// After esp_now_send(), instead of delay(100):
unsigned long t_wait = millis();
while (!messageSent && millis() - t_wait < 50) {
  delay(1);
}
```

This reduces the TX phase duration from approximately 99 ms to approximately 1.26 ms on average, saving around 98 ms of active time per cycle during which the WiFi radio would otherwise remain unnecessarily powered on.

Metric	Before	After
TX duration	∼99 ms	∼1.26 ms
TX energy	0.068 J	0.001 J
Energy per cycle	0.651 J	0.583 J
Estimated lifetime	2.02 days	2.22 days

Table 5: *Impact of the flag-based wait: TX duration drops by ∼98 ms, saving 0.067 J per cycle.*

It is worth noting that the callback signature used in this project is `const wifi_tx_info_t*`, which corresponds to the ESP32 Arduino framework version 3.x compiled via PlatformIO. The Wokwi web simulator still uses the older `const uint8_t*` signature from version 2.x, so the same code may not compile on the web interface without adjusting this parameter type.

7.4 Increasing the Deep Sleep Duration

The deep sleep duration $X = 4.5\,\text{s}$ was imposed by the challenge formula and is not a free design parameter in this implementation. However, from an application standpoint, a smart-home motion sensor does not need to report its status multiple times per second. Human activity in a room changes on a timescale of seconds to minutes, so a wake-up interval of 30 or 60 seconds would be entirely sufficient without any meaningful loss of responsiveness.

Since Deep Sleep is the most energy-efficient state by far, increasing its duration directly reduces the average power consumption proportionally. As an illustrative example, extending the sleep interval to 60 s, combined with the three improvements above, would bring the estimated lifetime from 3.2 to 4.75 days, approaching the theoretical maximum of approximately 5 days achievable if the node were always in Deep Sleep.

Metric	Sleep 4.5 s (spec)	Sleep 60 s (relaxed)
Energy per cycle	0.333 J	2.874 J
Estimated lifetime	3.2 days	4.75 days

Table 6: *Effect of extending Deep Sleep to 60 s on top of all other improvements. The higher per-cycle energy is offset by the much lower wake-up frequency, resulting in a significantly longer battery lifetime.*

It is worth clarifying why a longer sleep interval leads to a longer battery lifetime even though the energy consumed per cycle increases. What ultimately determines battery lifetime is the average power drawn over time, not the energy per cycle in isolation. The dominant cost in each duty cycle is the WiFi initialization phase, which draws approximately 632 mW for a fixed 196 ms regardless of how long the node sleeps afterward. By extending the sleep interval to 60 s, this fixed overhead is paid 13× less frequently and is therefore diluted over a much larger time window, reducing the average power significantly. The primary function of the node is also preserved: a 60-second sampling period is entirely adequate for a smart-home motion sensor.

7.5 Transmitting Only on Status Change

In the current implementation the node transmits a new message at every wake-up, regardless of whether the sensed conditions have changed since the previous cycle.

In a typical smart-home scenario, the state of a room remains constant for extended periods, so most transmissions carry redundant information and consume energy unnecessarily.

A more efficient strategy would be to store the last transmitted state in RTC memory, which survives Deep Sleep, and send a new message only when the current reading differs from the previous one. In scenarios where the environment is stable for long periods, this optimization could eliminate the vast majority of transmissions, dramatically extending the battery lifetime without changing the information available to the sink node.

7.6 Hardware Limitations of the ESP32 Architecture

It is worth noting that the ESP32, while widely used in IoT prototyping, is not optimized for ultra-low-power sensing applications. Its Deep Sleep current is acceptable (10–150 μA), but the 200 ms wake-up and radio initialization overhead is inherent to the architecture and cannot be reduced through software alone. For a production deployment, migrating to a more power-efficient platform would represent the most impactful long-term improvement.

8 Comparison Between Original and Improved Version

The three modifications described in the previous section (reducing the boot delay from 1000 ms to 10 ms, replacing the fixed `delay(100)` with a flag-based wait, and lowering the TX power to 2 dBm) are all compatible with the challenge constraints and are applied simultaneously in the improved code.

Their combined effect on the duty cycle is summarized in Table 7.

Phase	**Power**	**Time (orig.)**	**Time (impr.)**	**Energy (impr.)**
Boot delay	252 mW	1000 ms	10 ms	0.003 J
Sensor acquisition	238 mW	0.43 ms	0.43 ms	0.000 J
WiFi + ESP-NOW init	632 mW	196 ms	196 ms	0.124 J
Transmission	618 mW	99.3 ms	1.26 ms	0.001 J
Deep Sleep	46 mW	4500 ms	4500 ms	0.206 J
Total active time	—	≈1296 ms	≈208 ms	—
Active fraction	—	≈22%	≈4.4%	—
Energy per cycle	—	0.651 J	0.333 J	—
Estimated lifetime	—	2.02 days	3.20 days	—

Table 7: *Full comparison between the original and improved implementation. The power values are those measured from the CSV traces and are shared between the two versions except for TX, where 618 mW reflects the reduced 2 dBm setting. Deep Sleep power is identical in both versions.*

The most significant result is the reduction of the active fraction from approximately 22% down to approximately 4.4% of the total cycle.

Together, the three changes bring the energy per cycle from 0.651 J to 0.333 J, increasing the estimated battery lifetime from 2.02 to 3.20 days without any change to the node's sensing and communication behavior: the same two sensors are read, the same message format is used, and the same ESP-NOW broadcast protocol delivers the data to the sink node.

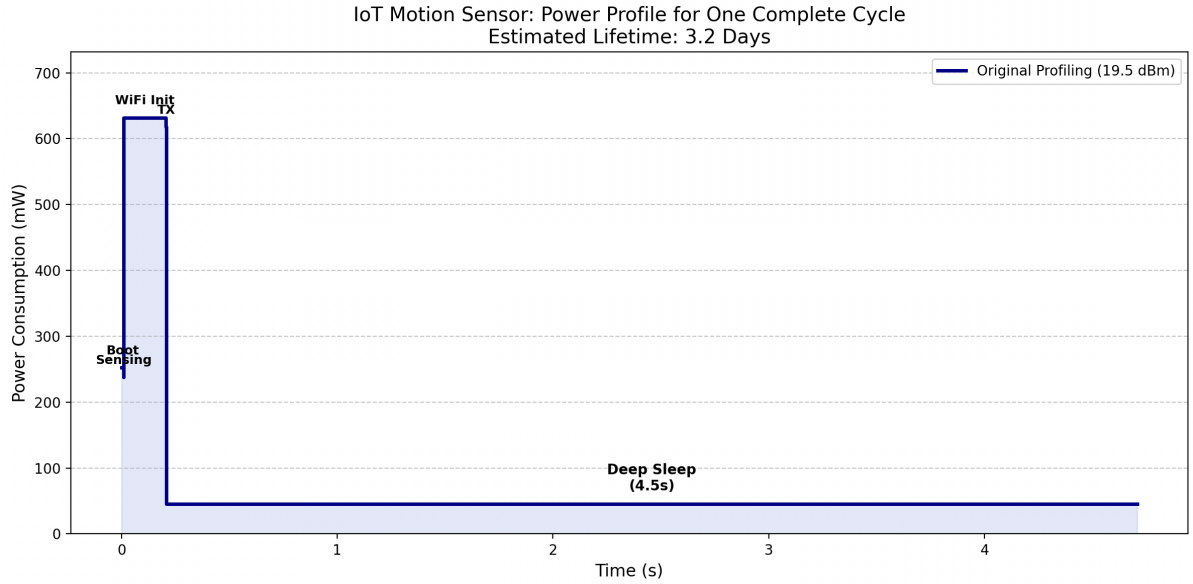


Figure 3: *Power consumption of the improved system over one duty cycle*

The power-vs-time graph makes the improvement immediately visible. In the original version the cycle opens with a 1-second plateau at the idle power level due to the boot delay, followed by the WiFi initialization peak and a 99 ms TX window. In the improved version the boot delay shrinks to 10 ms and the TX window drops to roughly 1.26 ms, so the active portion of the graph is dramatically shorter before the node drops back into Deep Sleep.

For completeness, Table 8 extends the comparison to include the scenario in which the Deep Sleep interval is also relaxed from 4.5 s to 60 s. Although this configuration goes beyond the challenge constraints, it illustrates the theoretical upper bound achievable through software-only optimizations on the current hardware platform. The active fraction in this case drops below 0.35%, and the estimated battery lifetime reaches 4.75 days, approaching the theoretical ceiling of approximately 5 days that would be achieved if the node were always in Deep Sleep.

Metric	**Original**	**Improved**	**+ Sleep 60 s**
Boot delay	1000 ms	10 ms	10 ms
TX duration	99.3 ms	1.26 ms	1.26 ms
TX power	688 mW	618 mW	618 mW
Deep Sleep	4500 ms	4500 ms	60 000 ms
Total active time	≈1296 ms	≈208 ms	≈208 ms
Active fraction	≈22%	≈4.4%	≈0.34%
Energy per cycle	0.651 J	0.333 J	2.874 J
Estimated lifetime	2.02 days	3.20 days	4.75 days

Table 8: *Full three-way comparison: original implementation, improved version (three software fixes within challenge constraints), and improved version with Deep Sleep extended to 60 s (relaxed constraint). The per-cycle energy increases in the third scenario because the sleep interval is 13× longer, but the average power and therefore the battery lifetime both improve significantly.*

9 Conclusion

This work presented the design, simulation, and energy analysis of a duty-cycled IoT motion sensor node based on the ESP32 platform. The implemented node satisfies all challenge requirements: it correctly reads motion and ambient light data through a PIR sensor and a photoresistor, formats the output string according to the specified syntax, and transmits it to the sink node via the ESP-NOW broadcast protocol. After each transmission, the board enters Deep Sleep for the interval $X = 4.5\,\text{s}$ derived from the team leader's person code, and wakes up again to repeat the cycle.

The energy analysis confirmed that wireless communication is by far the most demanding component of the duty cycle. The WiFi and ESP-NOW initialization phase alone draws over 630 mW for nearly 200 ms at every wake-up, and this cost is essentially fixed: it cannot be avoided as long as ESP-NOW is used, and it cannot be reduced through software tuning alone. This highlights a fundamental constraint of the ESP32 architecture for battery-powered applications.

Deep Sleep, on the other hand, is the key enabler of long battery lifetime. At roughly 46 mW, the power drawn during sleep is more than an order of magnitude lower than during active operation. Since the node spends the majority of its time in this state, even modest reductions in active time translate into meaningful improvements in overall efficiency. The three software optimizations applied in this work, eliminating the wasteful boot delay, replacing the fixed `delay(100)` with a flag-based wait, and lowering the transmission power to 2 dBm, together reduce the active fraction from approximately 22% to 4.4% and cut the energy per cycle from 0.651 J to 0.333 J, extending the estimated battery lifetime from 2.02 to 3.20 days.

These results demonstrate that even simple, zero-cost software changes can have a significant impact on the energy budget of a constrained IoT device, provided that the sources of waste are correctly identified through measurement rather than guesswork.